

The Feeney Triangle: A Generalised Recursive Triangle and an Investigation of Its Mathematical Properties

Dominic Feeney

July 29, 2026

Abstract

The Feeney Triangle is a recursively generated triangular array defined by a starting value, a fixed binary operator, and a fixed constant. Although its construction is simple, it exhibits a variety of mathematical behaviours that depend upon the chosen operator. This paper gives a formal definition of the Feeney Triangle and derives closed-form formulae for the addition, subtraction, multiplication, and division variants; the exponentiation variant is shown to remain only partially understood beyond its first column. Several structural properties of the triangle are examined, including the behaviour of its rows, columns, and diagonals, and the triangle is compared with existing recursive constructions, in particular Pascal's Triangle and the Bell Triangle. A possible interpretation of the triangle as a computational model is discussed; this interpretation is speculative and is offered as a direction for future work rather than an established result.

Contents

1	Introduction	2
2	Formal Definition of the Feeney Triangle	2
3	Basic Examples and First Observations	3
4	Why the Column Pattern Occurs	4
5	Closed-Form Formulae	4
5.1	Addition	4
5.2	Subtraction	5
5.3	Multiplication	5
5.4	Division	6
5.5	Exponentiation	6
6	Mathematical Properties	6
7	Relationship to Existing Mathematics	7
8	Interpretation as a Computational Model	8
9	Potential Applications	8

1 Introduction

Mathematics has a long history of discovering surprisingly rich structures from simple recursive rules. Well-known examples include Pascal's Triangle, the Bell Triangle, and Stern's Diatomic Array, each of which demonstrates how a straightforward recurrence can generate complex and unexpected behaviour. The Feeney Triangle belongs to this broad family of recursive triangular arrays, but differs in that its defining rule is not restricted to addition: the same recursive framework may be applied using any fixed binary operator together with a fixed constant.

The construction arose while attempting to build an intentionally complicated numerical pattern. During this process, the column increments of the resulting triangle were observed to follow consistent patterns that persisted even when the starting value, the constant, and the operator itself were changed. This suggested the behaviour was a consequence of the recursive structure itself rather than of any particular numerical example.

This paper gives a formal definition of the Feeney Triangle (Section 2) and illustrates its construction with a worked example (Section 3). Section 4 explains the origin of the observed column patterns informally, and Section 5 makes this rigorous with closed-form formulae for four of the five operators considered. Section 6 examines further structural properties, Section 7 compares the construction with existing recursive triangles, and Sections 8–9 discuss a speculative computational interpretation and potential applications. Section 10 lists open problems, and Section 11 concludes.

The aim of this paper is not to claim that every property of the Feeney Triangle has been discovered or proven, but to establish a rigorous foundation for the construction, document the properties derived so far, and identify directions for future investigation.

2 Formal Definition of the Feeney Triangle

The Feeney Triangle is a recursively generated triangular array whose entries are determined by three parameters: a starting value S , a fixed binary operator O , and a fixed constant K .

Definition 2.1 (The Feeney Triangle). Let $F(n, m)$ denote the entry in row n and position m , where rows and positions are numbered from 1 and $1 \leq m \leq n$. The Feeney Triangle is defined recursively by three equations:

$$F(1, 1) = S, \tag{1}$$

$$F(n, 1) = F(n - 1, 1) O K \quad (n > 1), \tag{2}$$

$$F(n, m) = F(n, m - 1) O F(n - 1, m - 1) \quad (1 < m \leq n). \tag{3}$$

Equations (1)–(3) completely determine the triangle. We refer to a fixed m , read down through increasing n , as *column* m ; and to a fixed $d = n - m$, read down at constant distance from the right-hand edge of each row, as *diagonal* d . Column 1 and diagonal 0 therefore denote the left and right edges of the triangle respectively.

Unlike many classical recursive triangles, the Feeney Triangle is not tied to a single operation: the same recursive structure can be generated using addition, subtraction, multiplication, division, exponentiation, or, in principle, any binary operator for which the recurrence is well-defined.

Changing the operator changes the behaviour of the triangle while leaving its recursive framework unchanged.

Remark on well-definedness. Equations (2)–(3) are only meaningful when O is defined on the values that actually arise. For division this requires every entry used as a divisor to be nonzero; here we restrict attention to $S, K > 0$, under which this is automatic. For exponentiation with non-integer exponents, the base must remain positive throughout the triangle; we likewise restrict to $S, K > 0$ in Section 5.5. These restrictions are stated explicitly wherever they are used below.

This generality is one of the distinguishing features of the Feeney Triangle. Rather than representing a single numerical object, it defines a family of related recursive triangles that share the same construction but exhibit different mathematical properties depending on the chosen operator.

3 Basic Examples and First Observations

The recursive definition is best understood through a simple example. Throughout this section, addition is used as the binary operator, with $S = 1$ and $K = 1$.

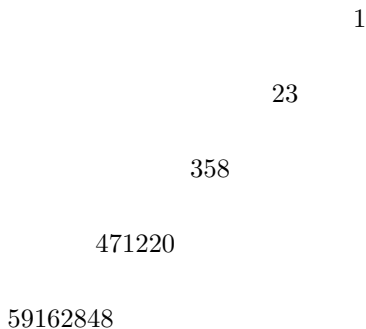


Figure 1: The addition Feeney Triangle with $S = 1$, $K = 1$, five rows.

Figure 1 shows the first five rows. Several observations can immediately be made. First, the triangle grows by one entry with each successive row, giving it its characteristic triangular shape. Secondly, every value is generated entirely from previously calculated values; no entry is chosen independently, so the entire structure is determined once S , O , and K are specified.

The most striking observation concerns the column increments. The increment along column 1 is determined by K . For the addition triangle, each subsequent column's increment doubles: column 1 increases by K at each row, column 2 by $2K$, column 3 by $4K$, and so on, as visible in Figure 1 (column 1: 1, 2, 3, 4, 5, common difference 1; column 2: 3, 5, 7, 9, common difference 2; column 3: 8, 12, 16, common difference 4). For multiplication, each new column's growth ratio is obtained by squaring the previous column's ratio. Under exponentiation, an analogous relationship holds for the first column considered on its own (Section 4); no corresponding pattern for later columns has yet been established, and this remains an open problem (Section 10). By contrast, subtraction and division behave very differently: after only two columns their entries collapse to fixed values, causing the triangle to stabilise.

At first glance these behaviours appear unrelated, yet they all arise from the same recursive rule; only the choice of operator changes. Explaining why these patterns emerge is the subject of Section 4, and Section 5 makes the explanation rigorous.

4 Why the Column Pattern Occurs

Consider the addition triangle with $S = 1$, $K = 1$. The increment along column 1 is the constant itself, $+1$. When column 2 is generated, the recursive rule combines two values that have both increased by $+1$ from one row to the next; since $+1 + +1 = +2$, the increment along column 2 becomes $+2$. Column 3 combines two increments of $+2$, giving $+2 + +2 = +4$. Repeating this argument shows that each successive column's increment is twice the previous column's increment. The doubling is therefore not accidental but follows directly from the recursive construction; Section 5.1 makes this precise and shows it holds for every S and K , not only $S = K = 1$.

The same reasoning extends to the other operators. For multiplication, the first column's growth ratio is set by K ; combining two identical multiplicative ratios gives $x \times x = x^2$, so each new column's ratio is the square of the previous column's. For example, in the triangle with $S = 2$, $K = 2$, column 2 has ratio 4 and column 3 has ratio $16 = 4 \times 4$.

Exponentiation requires more care. Using $(x^a)^b = x^{ab}$, repeated exponentiation multiplies the exponent governing column 1 by K at each row: if $F(n-1, 1) = S^{K^{n-2}}$ then $F(n, 1) = (S^{K^{n-2}})^K = S^{K^{n-1}}$. When $K = 2$ this multiplication happens to coincide with doubling, but for general K the correct statement is that the exponent is multiplied by K , not that it doubles. This describes only the row-to-row behaviour *within* column 1; no formula relating column 1 to column 2 or beyond has been found, and this remains the paper's principal open problem (Section 10).

Subtraction and division behave differently again. Column 1 under subtraction is arithmetic with common difference $-K$; combining it with itself one column over gives $x - x = 0$, so column 2 becomes the constant $-K$, and column 3 onward becomes exactly 0. Likewise under division, $x \div x = 1$, so column 2 becomes the constant $1/K$ and column 3 onward becomes exactly 1.

These observations explain why each operator produces a distinct family of column patterns while sharing the same recursive framework: the recurrence determines how values interact, while the operator determines how those interactions accumulate.

5 Closed-Form Formulae

For four of the five operators investigated — addition, subtraction, multiplication, and division — closed-form expressions were derived and each was checked computationally against the recursive definition across several thousand test cases spanning positive, negative, fractional, and large-magnitude values of S and K , with no discrepancies found within the tested ranges. (Because several of these variants grow doubly exponentially in m , the multiplication and exponentiation checks were restricted to parameter ranges avoiding floating-point overflow; see [4] for the relevant numerical caveats.) This computational agreement is supporting evidence, not a substitute for the proofs given below.

The exponentiation variant proved more difficult: a closed form was derived for the first column only, and no general expression is yet known for entries beyond it.

5.1 Addition

Theorem 5.1. Under addition, for every $1 \leq m \leq n$,

$$F(n, m) = 2^{m-2}(2S + (2n - m - 1)K).$$

Proof. Because the recurrence is linear in S and K — it only ever adds and combines terms, never multiplying S by K or squaring either — we may write $F(n, m) = A(n, m)S + B(n, m)K$, where A, B depend only on n, m , and solve for A and B separately.

Setting $K = 0$ collapses column 1 to the constant S ; taking $S = 1$ gives $F(n, m) = A(n, m)$ with $A(n, 1) = 1$ for all n . We claim $A(n, m) = 2^{m-1}$. By induction on m : the base case $A(n, 1) = 2^0 = 1$ holds by definition, and if $A(n, m-1) = A(n-1, m-1) = 2^{m-2}$ then

$$A(n, m) = A(n, m-1) + A(n-1, m-1) = 2 \cdot 2^{m-2} = 2^{m-1},$$

completing the induction and confirming A depends only on m .

Setting $S = 0$ gives $B(n, 1) = n-1$. For $m > 1$, B obeys $B(n, m) = B(n, m-1) + B(n-1, m-1)$. Guessing $B(n, m-1)$ is affine in n , say $B(n, m-1) = p_{m-1}n + q_{m-1}$ (true for $m = 2$, since $B(n, 1) = n-1$ has $p_1 = 1, q_1 = -1$), substitution gives

$$B(n, m) = 2p_{m-1}n + (2q_{m-1} - p_{m-1}),$$

again affine, confirming the guess for every m , with $p_m = 2p_{m-1}$ and $q_m = 2q_{m-1} - p_{m-1}$. Solving the first recurrence with $p_1 = 1$ gives $p_m = 2^{m-1}$. For the second, substituting the ansatz $q_m = a \cdot 2^m - m \cdot 2^{m-2}$ and matching $q_1 = -1$ gives $a = -\frac{1}{4}$, so $q_m = -(m+1)2^{m-2}$. Hence

$$B(n, m) = 2^{m-1}n - (m+1)2^{m-2} = 2^{m-2}(2n - m - 1).$$

Combining, $F(n, m) = A(n, m)S + B(n, m)K = 2^{m-1}S + 2^{m-2}(2n - m - 1)K = 2^{m-2}(2S + (2n - m - 1)K)$. $\square$

Setting $m = 1$ recovers the familiar arithmetic form $F(n, 1) = S + (n-1)K$, and the doubling of column increments described in Section 4 follows directly from the factor 2^{m-1} multiplying K .

5.2 Subtraction

Theorem 5.2. Under subtraction, $F(n, 1) = S - (n-1)K$, $F(n, 2) = -K$ for $n \geq 2$, and $F(n, m) = 0$ for $m \geq 3$.

Proof. Column 1 is arithmetic by direct unrolling of the recurrence, as in the addition case. For $m \geq 2$: suppose column $m-1$ is affine in n , say $F(n, m-1) = pn + q$. Then

$$F(n, m) = F(n, m-1) - F(n-1, m-1) = [pn + q] - [p(n-1) + q] = p,$$

a constant — in contrast to addition, where the analogous step stays affine and doubles, subtraction drops the degree by one, and the resulting constant equals the previous column's slope. Column 1 has slope $-K$, so column 2 is the constant $-K$. Applying the reduction once more: column 2, being constant, has slope 0, so column 3 is the constant 0, and every column thereafter, being the difference of a constant with itself, remains 0. $\square$

5.3 Multiplication

Theorem 5.3. For $S, K > 0$,

$$F(n, m) = S^{2^{m-1}} \cdot K^{2^{m-2}(2n-m-1)}.$$

Proof. For $S, K > 0$, every entry of the multiplication triangle is positive (an immediate induction), so $\log$ is defined throughout. Since $\log(ab) = \log a + \log b$, applying $\log$ to every entry produces an array obeying the addition recurrence with starting value $\log S$ and constant $\log K$. Substituting Theorem 5.1 with $S \rightarrow \log S$, $K \rightarrow \log K$ and exponentiating both sides gives the result. $\square$

5.4 Division

Theorem 5.4. For $S, K > 0$, $F(n, 1) = S/K^{n-1}$, $F(n, 2) = 1/K$, and $F(n, m) = 1$ for $m \geq 3$.

Proof. Identical to Theorem 5.3, using $\log(a/b) = \log a - \log b$ in place of $\log(ab) = \log a + \log b$, which links the logarithm of the division triangle to the subtraction triangle (Theorem 5.2) with starting value $\log S$ and constant $\log K$. $\square$

5.5 Exponentiation

Theorem 5.5. For $S, K > 0$, column 1 of the exponentiation triangle is $F(n, 1) = S^{K^{n-1}}$.

Proof. By induction on n , using $(x^a)^b = x^{ab}$: the base case $F(1, 1) = S = S^{K^0}$ holds, and if $F(n-1, 1) = S^{K^{n-2}}$ then $F(n, 1) = F(n-1, 1)^K = (S^{K^{n-2}})^K = S^{K^{n-1}}$. $\square$

Beyond the first column, no closed-form expression has yet been found; this remains the paper's principal open problem (Section 10).

6 Mathematical Properties

The behaviour of the Feeney Triangle depends strongly on the choice of operator, though every variant shares the same recursive framework. Columns behave regularly: for addition each fixed column forms an arithmetic sequence (Section 5.1), for multiplication each column forms a geometric sequence (Section 5.3), and for subtraction and division almost every column eventually becomes constant (Sections 5.2, 5.4). These regular column structures are exactly what makes the closed-form formulae of Section 5 derivable. In the exponentiation triangle only column 1 is understood, and it grows as a power tower rather than by a simple ratio or increment.

Rows behave very differently, exhibiting increasingly rapid growth for the addition, multiplication, and exponentiation variants. Theorem 5.1 shows the addition row entries carry a factor of 2^{m-1} , and this is not an incidental feature of the formula:

Proposition 6.1. Fix n and take the addition operator with $K \neq 0$. There is no polynomial P of any fixed degree such that $F(n, m) = P(m)$ for every m , uniformly as $n \rightarrow \infty$.

Proof. By Theorem 5.1, $F(n, m) = (\alpha m + \beta)2^m$ where $\alpha = -K/4$ and $\beta = (2S + (2n-1)K)/4$; note α depends only on K (and is nonzero whenever $K \neq 0$), while β depends on n , S , and K . The ratio of consecutive terms satisfies

$$\frac{F(n, m+1)}{F(n, m)} = \frac{\alpha(m+1) + \beta}{\alpha m + \beta} \cdot 2 \rightarrow 2 \quad \text{as } m \rightarrow \infty.$$

For any polynomial Q of degree $D \geq 1$, the leading term dominates for large m , giving $Q(m+1)/Q(m) \rightarrow 1$; for a constant ($D = 0$) the ratio is identically 1. Since $2 \neq 1$, no fixed-degree polynomial can match $F(n, \cdot)$ for large m . $\square$

This confirms the row behaviour is fundamentally exponential rather than polynomial, in the precise sense that no polynomial formula, of any fixed degree, can describe a row's entries as its length grows.

Another notable property is that the Feeney Triangle possesses no non-trivial symmetry: unlike Pascal's Triangle, which is symmetric about its centre, every entry here depends on the value immediately to its left and the value immediately above-left, producing an inherent left-to-right

asymmetry. This is visible already in row 4 of the canonical addition triangle, 4, 7, 12, 20, which is not a palindrome; reflecting the triangle does not preserve its values.

Despite these differences, every variant remains completely determined by only three parameters: S , K , and the chosen operator O .

7 Relationship to Existing Mathematics

Recursive triangular arrays appear throughout combinatorics, number theory, and algebra. The best-known example is Pascal's Triangle, whose entries are obtained by adding two values from the previous row. Other well-known constructions include the Bell Triangle, which generates the Bell numbers, and Stern's Diatomic Array [3].

The Feeney Triangle differs from these in two respects. First, it is *operator-independent*: classical triangles are generally defined using a single fixed operation, whereas addition, subtraction, multiplication, division, and exponentiation all produce valid members of the same family here while preserving the underlying recursive structure. Secondly, the recurrence itself differs from Pascal's: Pascal's Triangle combines two entries from the previous row, whereas the Feeney Triangle combines the value immediately to the left in the current row with the value immediately above-left in the previous row. This removes the symmetry present in Pascal's Triangle.

Among known recursive triangles, the Bell Triangle is structurally the closest. Both build each row from the previous row while progressing left to right, and for addition, the rule governing every entry after the first in a row has the same shape in both constructions. They differ in how each row begins: the Bell Triangle starts a new row by copying the previous row's last entry, whereas the Feeney Triangle starts a new row by combining the previous row's first entry with the external constant K .

Proposition 7.1. No choice of operator, starting value, or constant makes the Feeney Triangle identical, for all n , to the Bell Triangle.

Proof sketch. It suffices to compare column 1. It is a standard fact that the Bell numbers B_n (column 1 of the Bell Triangle) satisfy $B_{n+1}/B_n \rightarrow \infty$; this follows from their super-exponential growth rate, $\log B_n \sim n \log n$ (see [3]). A weaker version of this bound can be seen directly from the recurrence $B_{n+1} = \sum_{k=0}^n \binom{n}{k} B_k$, which is bounded below by the single term $\binom{n}{\lfloor n/2 \rfloor} B_{\lfloor n/2 \rfloor}$, and central binomial coefficients grow like $2^n/\sqrt{n} \rightarrow \infty$.

By contrast, column 1 of the Feeney Triangle is, under every operator considered here: affine in n (addition, subtraction) or geometric in n (multiplication, division), both of which have bounded $F(n+1, 1)/F(n, 1)$; or, under exponentiation (Theorem 5.5), a tower whose double logarithm is exactly linear in n , so that $F(n+1, 1)/F(n, 1)$ still fails to grow without the super-exponential acceleration that forces $\log B_n \sim n \log n$. In every case the growth rate of column 1 is inconsistent with that of the Bell numbers for all sufficiently large n , for any fixed parameters. $\square$

Although the two constructions share the same rule for extending a row, Proposition 7.1 shows this shared sub-structure is not enough to make them equivalent.

One particularly interesting observation is that diagonal 0 (the rightmost entries) of the canonical addition triangle ($S = 1, K = 1$) coincides with 1, 3, 8, 20, 48, 112, $\dots$, which is the Online Encyclopedia of Integer Sequences entry A001792, $a(n) = (n+2)2^{n-1}$ [1]. This sequence has several other combinatorial interpretations — for instance as the binomial transform of the natural numbers, and as the row sums of Bernoulli's triangle — so while the Feeney Triangle as a whole appears distinct from prior constructions, at least one of its diagonals coincides with an

already well-studied sequence. No identical recursive triangle matching the full construction has been identified.

For this reason the Feeney Triangle should not be viewed as an isolated object disconnected from existing mathematics, but as occupying its own place within the wider family of recursive triangular arrays [2]. Whether it represents an entirely new mathematical object or a previously overlooked variation of an existing construction remains open.

8 Interpretation as a Computational Model

Although developed as a mathematical construction, the recursive structure of the Feeney Triangle suggests a computational interpretation, offered here as speculation rather than an established result. Each entry combines two pieces of information: the value immediately to its left, representing a current state, and the value immediately above-left, representing information carried forward from an earlier stage. One way to interpret the triangle is as a sequence of processing stages: each row represents new data entering the system, and each column represents a further stage of processing.

This interpretation resembles the behaviour of prefix-sum (scan) algorithms: for addition, each row's entries are exactly the running prefix sums of the row above, offset by the row's own first entry. Similar accumulation patterns are used in parallel algorithms, graphics processing, and high-performance computing. No claim is made that the Feeney Triangle is identical to any existing hardware architecture or algorithm; it is offered as a mathematical perspective on recursive information accumulation, and whether this viewpoint leads to new algorithms remains an open question.

9 Potential Applications

The applications below are theoretical and are offered as directions for future research rather than established uses.

- **Algorithm analysis.** Since every entry is generated by combining previously computed values, similar recursive structures may be relevant when analysing algorithms that repeatedly update information.
- **Parallel computing.** Many parallel algorithms distribute work while repeatedly combining partial results; the recursive structure here may offer a framework for studying such propagation, though whether this yields practical improvements is unknown.
- **Education.** Because the same recursive definition applies to several different operations, the triangle offers a simple way to demonstrate how changing one component of a recursive system can produce very different behaviour, making it a useful example for teaching recursion, induction, and closed-form derivation.
- **State-based computational models.** The recurrence describes how new states arise from existing ones using only local information, suggesting a possible (but unverified) connection to such models.

Ultimately, the primary contribution of this paper may simply be as a subject of further mathematical research: many well-known mathematical objects began as investigations into simple patterns before finding applications later.

10 Open Problems

1. **Exponentiation beyond column 1.** No closed form is known for entries beyond the first column, and it is unknown whether one exists.
2. **General binary operators.** The recursive definition does not require the five operations studied here; it would be interesting to study its behaviour under other operators.
3. **A more general framework.** It is unknown whether the triangle admits a matrix formulation, a compact generating function, or another established representation that would simplify analysis.
4. **Further connections to existing mathematics.** Additional relationships to known recursive arrays beyond those discussed in Section 7 may exist.
5. **Computational interpretation.** The interpretations in Section 8 remain speculative; further work is needed to determine whether they have practical value.

11 Conclusion

This paper defined the Feeney Triangle, a recursive triangular array parameterised by a starting value S , a binary operator O , and a constant K , and investigated its behaviour under five choices of operator. The consistent column patterns that motivated the investigation were shown to follow directly from the algebraic properties of the chosen operator rather than from numerical coincidence, and closed-form formulae were derived for the addition, subtraction, multiplication, and division variants. The exponentiation variant remains only partially understood, and identifying its unresolved behaviour provides a clear direction for future work. The triangle was compared with existing recursive constructions, in particular Pascal's Triangle and the Bell Triangle, and while it shares structural features with both, it does not coincide with either. Several possible computational interpretations and applications were discussed; these remain speculative. This investigation should be regarded as a foundation for further study of the Feeney Triangle rather than as its conclusion.

Acknowledgements

I would like to acknowledge the assistance of the artificial intelligence systems ChatGPT, Claude, and Gemini during this investigation. These tools were used to discuss ideas, verify calculations, improve the presentation of mathematical arguments, identify relevant existing mathematics, and review drafts of this paper. The original concept of the Feeney Triangle, the direction of the investigation, the mathematical investigation, and the final decisions regarding the content of this paper remain my own.

References

- [1] OEIS Foundation Inc., *The On-Line Encyclopedia of Integer Sequences*, Sequence A001792, <https://oeis.org/A001792>.
- [2] R. L. Graham, D. E. Knuth, O. Patashnik, *Concrete Mathematics*, Addison-Wesley.

- [3] R. P. Stanley, *Enumerative Combinatorics, Volume 1*, Cambridge University Press.
- [4] D. Goldberg, “What Every Computer Scientist Should Know About Floating-Point Arithmetic,” *ACM Computing Surveys*, 1991.